odoo-loop user guide

Step by step: building Odoo 18 addons with AI agents. Spec first, tests locked by hash, then an implementation loop that may only stop when the judge (scripts/run_tests.sh) passes.

The core idea

AI agents are good at writing code, and just as good at claiming they are "done". This framework separates three things: **what** to build (the spec, written together with a human), the **proof** that it is right (tests, locked by hash before implementation starts), and **who decides** it is done (the judge script, not the agent). The agent may retry as often as it needs, but it cannot change the tests or the judge.

1	2	3	4	5
/spec	**/tests**	**Review + lock**	**/implement**	**Review + commit**
Agent + human	Agent	Human	Agent (loop)	Reviewer agent + human
specs/<addon>.md	drafts/<addon>/tests/	.tests.lock	addons/<addon>/	git commit

The last row is the output of each step. Step 3 and the final commit are always done by a human.

What is in the repo

Path	Purpose
CLAUDE.md (= AGENTS.md)	Project rules for the agent: definition of done, prohibitions, Odoo 18 conventions.
specs/<addon>.md	Spec and acceptance criteria (ACs). The source of truth.
drafts/<addon>/tests/	Draft tests written by the agent. Git-ignored; a human moves them after review.
addons/<addon>/	Addon code. The tests/ subfolder is locked for agents.
.tests.lock	SHA-256 hashes of every test file. Checked on every run_tests.sh run.
scripts/run_tests.sh	The judge: lint, install the addon in a clean test DB, run tests, give a verdict.
scripts/run_loop.sh	Outer loop across sessions for Claude Code (max. 8 rounds).
scripts/run_loop_opencode.sh	The same loop for OpenCode.
.claude/commands/	Slash commands: spec, tests, implement.
.claude/agents/odoo-reviewer.md	Diff reviewer subagent (read-only).
.claude/hooks/	block-dangerous.sh (blocks dangerous commands), stop-gate.sh (in-session loop).
.claude/settings.json	Allowed and denied files/commands for the agent, hook registration.
.opencode/, opencode.json	Equivalent commands, agent, and permissions for OpenCode.
.env.test	Odoo path, venv, PostgreSQL connection, test DB name. Never committed.
PROGRESS.md	The agent's iteration log; read at the start of every session or round.

0 Set up the environment

Human, once

a. Odoo 18 and a venv with the linters

The judge needs an Odoo 18 that runs from the command line, plus flake8 and pylint-odoo in the same venv. Check and install:

```
$VENV/bin/odoo --version                # must print: Odoo Server 18.0-...
$VENV/bin/python -m pip install flake8 pylint-odoo
```

b. The .env.test file

Copy the example, then adjust the paths for your machine:

```
cp .env.test.example .env.test
```

Replace `<venv>` with the path of your Odoo 18 venv. With a source checkout, use `ODOO_BIN="<venv>/bin/python <src>/odoo-bin"` and `ODOO_ADDONS_PATH="<src>/addons"` instead.

```
ODOO_BIN="<venv>/bin/odoo"
ODOO_ADDONS_PATH="<venv>/lib/python3.12/site-packages/odoo-18.0.<build>-py3.12.egg/odoo/addons"
# Venv bin folder that contains flake8 and pylint (leave empty if they are on PATH):
VENV_BIN="<venv>/bin"
PGHOST=localhost
PGPORT=5432
PGUSER=odoo
PGPASSWORD=odoo
TEST_DB=db_test_myaddon        # MUST contain the word "test"
TEST_HTTP_PORT=8169            # different from your dev Odoo on 8069
```

Common pitfall
Every value must be on its own line. If a newline goes missing, for example `ODOO_ADDONS_PATH="..."# Venv bin ...`, bash cannot read the file (*syntax error near unexpected token*) and run_tests.sh stops immediately. Check with: `bash -c '. ./.env.test && echo OK'`

c. PostgreSQL and git

- The PostgreSQL user in .env.test must be allowed to CREATE and DROP DATABASE. The judge drops and recreates TEST_DB on every run, and refuses any DB name that does not contain "test".
- The project must be a git repo (`git init`), because locked tests and the implementation are committed.
- Make sure `sha256sum` and `jq` are available. jq is used by the block-dangerous.sh hook.

1 Write the spec: /spec <addon> [description]

Agent + human

In Claude Code, run the slash command with the technical addon name. You can add the business process description directly:

```
/spec hotel_reservation
/spec petty_cash Staff request petty cash up to 5 million, a supervisor approves
```

If information is missing, the agent asks **once** about four things: the business process (who does what, the state flow, numeric limits, who approves), user roles, module depends, and what is out of scope. After you answer, it writes `specs/<addon>.md` with:

- Goal, Manifest (version 18.0.1.0.0, depends, LGPL-3), Models (fields, types, defaults)
- **Acceptance criteria**: at most 8, numbered AC-01..AC-08, each one behavior that a TransactionCase can test, with clear numeric limits
- UI, **Security** (group xml ids, ir.model.access, record rules), Out of scope, Definition of Done
- **AC-to-test mapping**: a table of AC, test name, scenario, expected assertion
- **Open questions**: anything ambiguous, so the agent does not guess

> **The human's job in this step**
> Read the ACs and answer the Open questions. If an answer changes behavior, update the spec **before** moving on. Once the tests are locked, changing behavior means changing the tests and the lock.

2 Write draft tests: /tests <addon>

Agent

```
/tests hotel_reservation
```

The agent reads the spec and writes `drafts/<addon>/tests/__init__.py` and `test_<addon>.py`. It writes no addon code and does not touch addons/*/tests/. Draft rules:

- One TransactionCase class with `@tagged('post_install', '-at_install')`
- setUpClass creates one user per role with the groups from the spec's Security section
- At least one test per AC, named `test_acXX_short`, in AC order
- "May not" rules are tested with assertRaises(ValidationError / UserError / AccessError)
- Limits are tested EXACTLY at the limit and one past it; sensitive actions are tested through `.with_user(regular_user)`
- No skips of any kind, and flake8 clean (max-line-length 120)

If the spec is missing, empty, or has no mapping table, the agent **stops** and names what is missing instead of inventing ACs.

Lint the drafts by hand (the judge does the same later):

```
$VENV_BIN/flake8 --max-line-length=120 --per-file-ignores="__init__.py:F401" \
    drafts/hotel_reservation/tests
$VENV_BIN/pylint --rcfile=.pylintrc -d manifest-required-author,missing-readme \
    drafts/hotel_reservation/tests
```

3 Review, move, and lock the tests

Human

Read the drafts and make sure every test checks a real outcome (state, field value, record count). Then run:

```
mkdir -p addons/hotel_reservation/tests
cp drafts/hotel_reservation/tests/*.py addons/hotel_reservation/tests/
scripts/run_tests.sh hotel_reservation
#   new addon: MUST fail "__manifest__.py is missing" (exit 7)
find addons/*/tests -name '*.py' -exec sha256sum {} + > .tests.lock
git add -A && git commit -m "test: acceptance tests hotel_reservation (locked)"
```

Check the lock at any time with `sha256sum -c .tests.lock`. Every line must say OK.

> **Why must run_tests.sh fail first?**
> It proves that the tests are found and that the judge can return FAILED. A test that passes before any code exists is not testing anything.

Important
The find command locks the tests of **all** addons at once, so older addons stay protected. Only a human regenerates .tests.lock; settings.json forbids the agent from doing it.

4 Implement until the judge passes

Agent (loop)

There are three ways to run the loop. All three use the same judge, and done means **exit 0 and "=== PASSED ===" printed**.

Option A: interactive, in one session

```
/implement hotel_reservation
```

The agent works in stages (skeleton, models, workflow, security, views), runs run_tests.sh after each stage, and writes a summary to PROGRESS.md. You can watch and interrupt.

Option B: in-session loop (Stop hook)

```
LOOP_MODE=1 MODULE=hotel_reservation claude
> /implement hotel_reservation
```

Whenever the agent wants to stop, stop-gate.sh runs the judge. If it fails, the agent is forced to continue with the last 60 log lines. After at most 8 attempts, a human takes over. **MODULE is required**; it defaults to petty_cash.

Option C: outer loop across sessions (headless)

```
scripts/run_loop.sh hotel_reservation            # Claude Code
scripts/run_loop_opencode.sh hotel_reservation   # OpenCode
```

Each round runs the judge, saves the result to last_run.txt, then opens a fresh agent session that reads CLAUDE.md, the spec, PROGRESS.md, and last_run.txt to make one small fix. At most 8 rounds. For Claude, each round is capped at 30 turns and USD 3. For OpenCode, each round is capped at 20 minutes.

	/implement (A)	run_loop.sh (C)
Memory	One long conversation; the agent remembers everything it tried	Every round starts fresh; memory lives only in PROGRESS.md and last_run.txt
Interaction	You can watch, interrupt, and answer questions	Unattended; you can walk away
Best for	The first big implementation, which needs planning and full context	Small repeated fixes once most of the code exists, or overnight runs

Stop rules for the agent
If the same error appears 3 iterations in a row, or a test looks wrong, the agent must stop, write its hypothesis in PROGRESS.md, and ask for human review. It must never game the tests.

5 Review the diff, then commit

Reviewer agent + human

After PASSED, ask the reviewer subagent to check the diff before committing:

```
Use the odoo-reviewer agent to review the hotel_reservation diff
```

The reviewer is read-only and reports findings by severity:

- every AC has an implementation and a meaningful test
- no features outside the scope of the spec
- security: access rules, record rules, group checks in sensitive methods
- Odoo 18 conventions: <list>, no attrs/states, <chatter/>
- tricks that make tests pass (hardcoding, skips)

Fix the findings, run the judge once more, then commit. `git push` is always done by hand.

Reading the judge's verdict

scripts/run_tests.sh <addon> runs its checks in order and stops at the first failure:

Exit	Message	Meaning and action
0	=== PASSED ===	All lint checks and tests passed, and at least 1 test really ran. Done.
2	no tests in addons/<addon>/tests yet	Drafts not moved yet. Do step 3.
4	test files changed since they were locked	Hashes do not match .tests.lock. Restore the test files, or re-lock (humans only).
5	skip found in tests/	A skipTest/@skip/skipIf is present. Remove it.
6	TEST_DB must contain the word 'test'	Guard so no other DB gets dropped. Fix .env.test.
7	__manifest__.py is missing	Normal before implementation. Move on to /implement.
3	FAILED: flake8 / pylint-odoo	Lint violations in the addon. See the printed lines.
1	=== FAILED (exit N) ===	Install or tests failed: ERROR/FAIL/Traceback lines are printed from test.log. Also shown when 0 tests ran, or the DB cannot be dropped.

Safety rails

Layer	What it protects
.claude/settings.json (deny)	The agent may not edit addons/*/tests/, .tests.lock, scripts/, .pylintrc, or .env.test, may not read .env.test, and may not run rm -rf, git push, or dropdb. Deny rules match by path pattern, so a scripts/ folder anywhere in the project is protected too.
.claude/settings.json (allow)	Editing addons/ and PROGRESS.md, run_tests.sh, and git status/diff/add/commit without asking.
hooks/block-dangerous.sh	Blocks any Bash command containing rm -rf, git push, drop database, dropdb, --force, or db_prod anywhere in the command, even "which dropdb". DB access goes through run_tests.sh only.
hooks/stop-gate.sh	With LOOP_MODE=1, the agent may not stop before the judge passes (max. 8 times).
.tests.lock + judge	Changed, skipped, or zero tests always FAIL.
opencode.json	Equivalent permissions for OpenCode: edits allowed, other bash must ask, dangerous commands denied.

When tests must change after locking

For example, because an Open question in the spec was answered later. Do it by hand, not through the agent:

```
# 1. update specs/<addon>.md
# 2. edit addons/<addon>/tests/*.py (or rerun /tests and copy the new drafts)
find addons/*/tests -name '*.py' -exec sha256sum {} + > .tests.lock
git add -A && git commit -m "test: update acceptance tests <addon> (locked)"
```

Troubleshooting

Symptom	Cause and fix
/tests stops: spec not found	The addon name in the command does not match the spec file or addon folder. Always run /spec first, with the same technical name.
/tests stops: spec is empty	The spec file is 0 bytes. The agent refuses to invent ACs. Fill it with /spec.
flake8/pylint not found	VENV_BIN in .env.test points to a venv that does not exist. Point it to the right Odoo 18 venv.
.env.test syntax error	A newline is missing between a value and a comment. One value per line.
DB check blocked by hook	Any command containing the word dropdb is blocked. This is intentional; the DB connection is tested through scripts/run_tests.sh.
Agent cannot write scripts/ or .pylintrc	Intentional (settings.json deny). A human edits these files.

Command summary

```
/spec <addon> [description]                    # 1. spec + ACs (answer questions)
/tests <addon>                                 # 2. draft tests in drafts/<addon>/tests/
mkdir -p addons/<addon>/tests                  # 3. human: review, copy, lock
cp drafts/<addon>/tests/*.py addons/<addon>/tests/
scripts/run_tests.sh <addon>                   #    must FAIL: manifest is missing
find addons/*/tests -name '*.py' -exec sha256sum {} + > .tests.lock
git add -A && git commit -m "test: acceptance tests <addon> (locked)"
/implement <addon>                             # 4. or scripts/run_loop.sh <addon>
scripts/run_tests.sh <addon>                   #    done = exit 0 + "=== PASSED ==="
                                               # 5. odoo-reviewer, then commit
```